Project Proposal(2.0) – WSSOA (IT644)

Autumn 2026

Project Title :

PriceWise - Compare, Analyze, Shop Smarter

A web-based AI-Powered Smart Shopping Assistant

Date of Submission: **16/08/2026**

Submitted By:

Sr. No.	Student Name	Student ID
1	Diya Jain	202512014
2	Piyush Tanwani	202512021
3	Divyesh Dandwani	202512043
4	Harshil Dodwani	202512044
5	Vivek Dhanwani	202512045
6	Yash Gangwani	202512048
7	Aman Rohera	202512049
8	Priya Shah	202512100

Submitted To:

Prof. Jayprakash Lalchandani
Course Instructor - WSSOA (IT644)
DAU, Autumn 2026

Project Title and Short Description

Title :

PriceWise - A Shopping Intelligence Platform

Description :

PriceWise Shopping Intelligence Platform is an AI-powered platform designed to improve the online shopping experience by combining product discovery, comparison, price intelligence, and personalized shopping insights in a single platform.

With the rapid growth of e-commerce, users often find it difficult to choose the right product from thousands of available options across multiple online stores. They need to manually visit different platforms to compare specifications, check prices, analyze reviews, and track price changes. This process is time-consuming and does not always provide enough information to make an informed choice.

PriceWise solves this problem by providing a unified platform where users can search products, explore detailed specifications, compare similar products, and view prices from multiple stores such as Amazon, Flipkart, Croma, and other available sources. The platform also maintains historical price data and provides interactive price trend analysis to help users understand product pricing patterns.

Using Artificial Intelligence and Machine Learning, PriceWise provides advanced features such as future price prediction, personalized product recommendations, smart alternative suggestions, and price-drop alerts. These features help users discover suitable products, identify better deals, and gain meaningful insights before purchasing.

The goal of PriceWise is to transform traditional online shopping into a smarter, datadriven experience by combining product comparison, market analysis, and AI-powered recommendations on a single platform.

Application Type

Application Category: Web Application:

PriceWise is a web-based application that provides users with a centralized platform for product discovery, comparison, price tracking, and AI-powered shopping insights.

Reasons for Choosing Web Application:

- **Easy Accessibility:** Users can access PriceWise through any web browser without installing additional software.
- **Better Data Visualization:** The platform involves displaying product comparisons, price trends, historical graphs, and analytical insights, which can be effectively represented through a web interface.
- **Cross-Platform Support:** Users can access the application from different devices such as laptops, tablets, and smartphones.
- **Easy Maintenance and Updates:** New features, improvements, and updates can be deployed easily without requiring users to manually update the application.
- **Scalability:** The web architecture allows smooth integration with backend services, databases, and AI/ML models as the platform expands.
- **Integration Capability:** The application can easily connect with external APIs, data sources, notification systems, and machine learning services.

Motivation / Problem Statement

Problem Statement:

The rapid growth of e-commerce has made online shopping more convenient but has also increased the complexity of choosing the right product. Users have access to thousands of products across multiple platforms, making it difficult to compare options and identify the best choice.

Currently, users need to manually visit different websites to:

- Search for products across multiple platforms.
- Compare product specifications and features.
- Check prices from different online stores.
- Analyze whether the current price is reasonable.
- Track historical price changes.
- Find better alternatives within their budget.

This process is time-consuming and often requires users to collect information from multiple sources before making a purchase decision. Existing shopping platforms mainly focus on selling products and provide limited insights about price trends, future price changes, and personalized recommendations.

Motivation:

The motivation behind PriceWise is to create a unified platform that simplifies the online shopping process by combining product information, price comparison, historical analysis, and AI-powered insights in one place.

The platform aims to:

- Reduce the effort required to research products across multiple websites.
- Help users compare products based on features, specifications, and prices.
- Provide insights into price trends and possible future price movements.
- Recommend suitable products based on user preferences and requirements.
- Help users discover better alternatives and available deals.

By combining data analytics and machine learning, PriceWise aims to make online shopping more intelligent, transparent, and efficient.

Target Users and Use Cases

Use Cases :

Use Case	Description
Product Search	Search products using name, category, and brand filters.
Product Comparison	Compare specifications, features, ratings, and prices of multiple products.
Price Comparison	Compare prices across different online stores to find better deals.
Price Analysis	View historical price trends and analyze price changes.
AI Insights	Get price predictions, recommendations, and smart alternatives.
Price Alerts	Receive notifications when product prices drop or reach the desired value.

Target Users:

Target Users	Use Cases
Online Shoppers	Search products, compare options, check prices, and find better deals before purchasing.
Budget-Conscious Users	Discover products within budget and find cost-effective alternatives.
Students & Professionals	Find suitable products based on requirements such as performance, features, and price.
Technology Enthusiasts	Track new products, analyze price trends, and monitor market changes.

Scope and High-Level Features

Scope of the Project:

The project aims to develop a unified shopping platform that helps users discover products, compare options, analyze prices, and receive intelligent shopping insights.

The system reduces the need to visit multiple shopping websites by bringing product details, price comparison, historical trends, and recommendations together in one place. The scope includes product search, price tracking, AI-based analysis, and personalized shopping features.

High-Level Features:

1. Product Discovery

- Search products by name, category, and brand.
- View specifications, images, and ratings.

2. Product Comparison

- Compare products based on features, specifications, and prices.
- Highlight key differences between products.

3. Price Intelligence

- Compare prices across multiple stores.
- Track historical price trends using graphs.

4. AI-Powered Insights

- Predict future price trends.
- Provide smart recommendations and alternative product suggestions.

5. User Personalization

- Wishlist and watchlist management.
- Personalized recommendations based on user preferences.

6. Price Alerts

- Notify users about price drops and better deals.

7. Data Analytics

- Analyze product and pricing data to generate useful shopping insights.

Product Identity Matching for Cross-Store Price Comparison

Cross-store price comparison only works if PriceWise can confirm that a listing on Amazon, a listing on Flipkart, and a listing on Croma are genuinely the same product. Since each store writes its own title, images, and description, this identity resolution is a core engineering problem, not an assumption - the proposal previously left this unexplained; this section clarifies the approach.

Matching Pipeline

- Stage 1 - Attribute normalization: extract brand, model number, and key specs (RAM, storage, size, colour) from each listing's title and spec table into a normalized schema.
- Stage 2 - Deterministic matching: where a GTIN/EAN/UPC or manufacturer part number (MPN) is available from the store feed, match directly on that identifier - the most reliable signal.
- Stage 3 - Fuzzy / semantic matching: when no shared identifier exists, compute similarity between normalized titles using TF-IDF cosine similarity plus attribute-overlap scoring (brand + model + spec match) to generate a confidence score.
- Stage 4 - Canonical Product ID: matches above a confidence threshold are merged under one internal PriceWise Product ID; each store listing becomes a linked "offer" under that canonical product.
- Stage 5 - Manual review queue: low-confidence matches (e.g., 0.55–0.80 similarity) are queued for a lightweight admin review step rather than auto-merged, to avoid comparing two different products by mistake.

This matching logic sits in the backend as a dedicated "Product Matching Service" that runs whenever new listings are ingested from the data collection pipeline (see Section 7), and re-runs periodically to catch re-matches as store catalogues change.

API Integration & Data Configuration

Live, real data sources are used end-to-end - no dummy/mock data is used beyond isolated unit tests. The following concrete sources power product and price data:

Store / Source	Integration Method	Data Retrieved
Flipkart	Flipkart Affiliate API (official REST API, requires affiliate registration)	Product catalogue, live price, availability, ratings
Amazon	Amazon Product Advertising API (PA-API 5.0) via official SDK	Product details, live price, offer listings
Croma & other stores without a public API	Compliant, rate-limited scraping service (Python + Scrapy/BeautifulSoup) respecting robots.txt and store ToS, refreshed on a schedule	Product listing, current price, stock status
Historical price tracking	Internal PriceWise Price-History service - snapshots prices from the above sources on a fixed interval and stores them in PostgreSQL/time-series table	Time-stamped price points per canonical product

Configuration & reliability practices:

- API keys/secrets are stored in environment variables and never committed to the repository; separate sandbox and production keys are used for dev and deployed environments.
- Responses are cached in Redis (with a short TTL) to stay within API rate limits and reduce redundant scraping load on store websites.
- A fallback/retry policy handles API downtime - if a store source is temporarily unavailable, the last known price is shown with a "last updated" timestamp rather than a broken UI.
- During development, sandbox/test API credentials are used strictly for functional testing against real endpoints - not hard-coded dummy JSON - so the integration logic is validated against actual API contracts before go-live.

Price Notification System

Users can watchlist a product and set a target price (or opt into "any price drop") from the product page. The notification flow is as follows:

- A scheduled backend worker (BullMQ job queue on Redis) re-checks tracked products' current prices against the Price-History service at a fixed interval.
- When a tracked product's price drops below the user's target (or drops at all, for drop-alerts), an event is pushed into the notification queue.
- Delivery channels: email (via SendGrid/Nodemailer transactional email) and in-app notifications by default; Firebase Cloud Messaging (FCM) push notifications as an optional stretch feature.
- Each notification includes the product, old price, new price, percentage drop, and a direct comparison link so the user can act immediately.
- Users manage alert preferences (channel, frequency, mute) from a notification settings page — avoiding alert fatigue.

Technology Stack

Category	Technologies
Frontend	React.js, Tailwind CSS, Shadcn/UI, Recharts / Chart.js
Backend	Node.js / Python
Database	PostgreSQL
Cache / Queue	Redis, BullMQ (for scheduled price checks & notifications)
AI/ML	Python, Pandas, NumPy, Scikit-learn, Prophet / XGBoost
ML API Service	FastAPI
Authentication	JWT, Google OAuth (optional)
External APIs	Flipkart Affiliate API, Amazon PA-API 5.0, compliant scraping service for other stores
Notifications	SendGrid / Nodemailer (email), Firebase Cloud Messaging (push, optional)
API Communication	REST APIs, Axios
Deployment	Vercel (Frontend), Render / Railway / AWS (Backend & ML Service), Neon PostgreSQL (Database)
Version Control / PM	Git, GitHub, Jira

Roles and Responsibilities

Each team member is assigned as the sole owner of specific tasks so accountability is unambiguous no task has two owners. Planned effort is approximately 200 hours per member across the 16-week project (≈12–13 hrs/week), based on the individual timelines in the Project Plan. Where members support each other's work, the primary owner is the one graded on that deliverable; supporting members are noted separately.

Team Summary

Member	Primary Role	Planned Effort
Piyush Tanwani	AI/ML Engineer & Database Architect	200 hrs
Divyesh Dandwani	Frontend Lead & Backend API Integrator	200 hrs
Diya Jain	Backend Support, API Testing & Documentation	200 hrs
Vivek Dhanwani	Data Engineer & Database Support	200 hrs
Yash Gangwani	Authentication & Recommendation Engine Developer	200 hrs
Aman Rohera	Backend & External API Integration Specialist	200 hrs
Harshil Dodwani	UI/UX Designer & Frontend-AI Integration	200 hrs
Priya Shah	QA Lead & AI Model Testing	200 hrs
Total	**8 members**	**1,600 hrs**

Individual Contribution Detail

Piyush Tanwani (202512021)

Primary Role: AI/ML Engineer & Database Architect

Owns the system architecture and database design, then leads the price-prediction model end to end from training through backend integration.

Task	Deliverable / Notes	Effort (hrs)
Planning & requirement gathering	Contributes to project scope and user stories	10
System architecture design	Defines overall service architecture and data flow	20
Database schema design	Canonical product, offers, price-history tables	25
Database setup & indexing	PostgreSQL setup, indexes for query performance	15
Price Prediction model development	Prophet/XGBoost model on historical price data	55
AI–Backend integration	Exposes prediction model via FastAPI to core backend	30
Model evaluation & tuning	Accuracy testing, hyperparameter tuning	25
Report & presentation	Final documentation and demo contribution	20
Total Individual Effort	Sole owner of every task listed above (one-person-one-task ownership)	200

Divyesh Dandwani (202512043)

Primary Role: Frontend Lead & Backend API Integrator

Owns the frontend build from wireframe to shipped UI, and independently owns the core backend APIs that power search and comparison.

Task	Deliverable / Notes	Effort (hrs)
UI wireframes	Low-fidelity wireframes for all core screens	10
High-fidelity UI mockups	Final visual design in Figma	15
API specification design	REST contract for frontend–backend communication	15
Frontend project setup	React + Tailwind + Shadcn scaffolding	10
Frontend components	Search, comparison, product detail, dashboard UI	45
Core backend APIs	Product & comparison endpoints (Node.js/Python)	30
Search & comparison feature	End-to-end feature: query → matched offers → UI	25
Price history chart integration	Recharts/Chart.js visualization of price trends	15
UI/UX testing & bug fixing	Cross-browser/device testing, defect resolution	20
Performance optimization	Frontend bundle & rendering optimization	15
Total Individual Effort	Sole owner of every task listed above (one-person-one-task ownership)	200

Diya Jain (202512014)

Primary Role: Backend Support, API Testing & Documentation

Supports backend development for assigned modules, handles API testing and documentation, and contributes to integration and quality testing.

Task	Deliverable / Notes	Effort (hrs)
Backend Support	Assist with assigned backend modules and API integration	30
API Testing	Test REST APIs using Postman and validate responses	35
API Documentation	Prepare and maintain API documentation using Postman/Swagger	30
Integration Testing	Test communication between frontend, backend and services	30
Bug Tracking & Verification	Record, reproduce and verify resolved issues	25
Technical Documentation	Prepare setup guides, API notes and project documentation	25
Final Testing Support	Support system testing and final verification	15
Report & Presentation	Documentation and presentation contribution	10
Total Individual Effort	Assigned tasks and supporting responsibilities	200

Vivek Dhanwani (202512045)

Primary Role: Data Engineer & Database Support

Owns the data-collection pipeline that feeds the entire platform — web scrapers, price APIs, and the historical price data store.

Task	Deliverable / Notes	Effort (hrs)
Planning & requirements	Data-source feasibility review	10
Database schema support	Collaborates on schema for offers/prices	15
Backend & DB setup support	Assists initial infra setup	15
Web scrapers	Compliant scrapers for stores without public APIs (e.g., Croma)	55
Data collection scripts	Scheduled ingestion jobs, data cleaning	35
Price APIs integration	Consumes Flipkart/Amazon APIs into normalized schema	30
Price history pipeline	Snapshot service feeding the price-history table	25
Report & presentation	Documents data pipeline design and coverage	15
Total Individual Effort	Sole owner of every task listed above (one-person-one-task ownership)	200

Yash Gangwani (202512048)

Primary Role: Authentication & Recommendation Engine Developer

Owns the User Management module end to end, and independently owns the Recommendation Engine — one of the platform's core AI features — from design through tuning.

Task	Deliverable / Notes	Effort (hrs)
Planning & requirements	User-journey and personalization requirements	10
JWT / OAuth authentication support	Implements auth middleware alongside Diya	15
User Management module	Sole owner: profile, preferences, wishlist/watchlist logic	35
Recommendation Engine — design	Feature selection, collaborative/content-based approach	25
Recommendation Engine — implementation	Sole owner: model build using Scikit-learn	45
Price-drop alert logic	Rules engine deciding when a drop qualifies as alert-worthy	25
Model evaluation & tuning	Precision/recall tuning of recommendations	25
Report & presentation	Documents recommendation methodology	20
Total Individual Effort	Sole owner of every task listed above (one-person-one-task ownership)	200

Aman Rohera (202512049)

Primary Role: Backend & External API Integration Specialist

Owns the integration layer connecting PriceWise to external retailer data sources, and the search & comparison performance work.

Task	Deliverable / Notes	Effort (hrs)
Architecture & API planning	Defines external-API integration contracts	15
Backend setup support	Assists core backend scaffolding	10
Core backend APIs	Offer aggregation endpoints	30
Data collection scripts support	Works with Vivek on ingestion jobs	20
Price APIs — external integration	Sole owner: Flipkart Affiliate API & Amazon PA-API clients	45
Search & comparison logic	Ranking and filtering of matched offers	30
AI–backend integration support	Connects prediction/recommendation outputs to API layer	20
Performance optimization	API response caching and rate-limit handling	15
Report & presentation	Documents API integration approach	15
Total Individual Effort	Sole owner of every task listed above (one-person-one-task ownership)	200

Harshil Dodwani (202512044)

Primary Role: UI/UX Designer & Frontend–AI Integration

Owns visual/UX design end to end and is the sole owner of surfacing AI outputs (predictions, recommendations, price graphs) in the UI.

Task	Deliverable / Notes	Effort (hrs)
Planning & requirements	UX requirements and user flows	10
UI wireframes	Collaborates with Divyesh on core wireframes	10
UI mockups	Detailed visual design system	15
Frontend setup support	Assists initial React scaffolding	10
UI components	Reusable design-system components	30
Price prediction UI	Sole owner: displays forecast with confidence range	20
Recommendations UI	Sole owner: "Recommended for you" & alternatives UI	20
Price graphs visualization	Historical trend charts (Recharts/Chart.js)	20
UI/UX testing & bug fixing	Usability testing and design QA	20
Technical documentation	Design system & component documentation	15
Deployment support (Web & ML)	Assists Diya with frontend/ML deploy verification	30
Total Individual Effort	Sole owner of every task listed above (one-person-one-task ownership)	200

Priya Shah (202512100)

Primary Role: QA Lead & AI Model Testing

Owns quality assurance across the platform and independently owns model-evaluation testing for the AI features.

Task	Deliverable / Notes	Effort (hrs)
Planning & requirements	Test-plan and acceptance-criteria definition	10
Price prediction support	Validates model output against real price data	20
Recommendations support	Validates recommendation relevance	20
Unit & integration testing	Sole owner: platform-wide test suite execution	45
Model evaluation	Sole owner: accuracy/precision metrics for all ML features	35
Technical documentation	Test reports and QA documentation	20
Frontend deployment support	Pre-release verification	15
Backend deployment support	Pre-release verification	15
ML deployment support	Validates deployed model matches tested version	20
Total Individual Effort	Sole owner of every task listed above (one-person-one-task ownership)	200

Timeline with Milestones

Week	Milestone	Activities
Week 1	Project Proposal	Finalize project scope, team roles, and submit the proposal.
Week 2	Planning & Requirements	Prepare project plan, gather requirements, create user stories and initial wireframes.
Week 3	System Design	Design database, system architecture, APIs, and UI mockups.
Week 4–7	Core Development	Develop frontend, backend APIs, authentication, and product management modules.
Week 8–10	Feature Integration	Implement product comparison, price tracking, historical graphs, and AI recommendation features.
Week 11	Mid-Sem Demo	Demonstrate a working prototype with core functionalities.
Week 12–14	Testing & Refinement	Perform testing, fix bugs, optimize performance, and complete documentation.
Week 15	Deployment	Deploy the application and integrate all modules.
Week 16	Final Submission	Submit the final project report and present the completed application.